Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

# ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

## Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

# Лабораторная работа 5

## Схема данных и справочник в Supabase

Продолжите проект четвёртой лабораторной. Создайте схему приложения в PostgreSQL, перенесите справочник из учебного JSON в Supabase и подключите его к уже работающей форме. Интерфейс сохранит состояния загрузки, ошибки, пустого результата и повторной попытки, но будет получать значения из реальной базы данных.

### Цель работы

Научиться описывать структуру данных через SQL, связывать таблицы внешним ключом и управлять доступом через права PostgreSQL и Row Level Security. Подключить Supabase к React через переменные окружения и заменить источник данных, не меняя пользовательский сценарий формы.

### Обязательный результат

В Supabase созданы 2–3 таблицы выбранной темы, среди них основная сущность и связанный справочник. Схема содержит первичные и внешние ключи, обязательные поля и предметные ограничения. Публичный справочник заполнен из seed.sql, доступен браузеру только для чтения и используется формой. Личные таблицы включают owner_id, но закрыты от клиентских ролей до ЛР 6.

Основные записи из ЛР 3 пока остаются в состоянии App и после перезагрузки возвращаются к демонстрационному набору. В этой работе постоянным становится только справочник. Не переносите локальные задачи в БД и не открывайте анонимную запись «на один этап». В ЛР 6 появятся Supabase Auth и безопасное хранение личных записей.

## Теоретическая справка

### Таблицы, ключи и ограничения

Таблица хранит записи одного вида, а столбцы задают их свойства. Primary key однозначно определяет строку. Foreign key связывает значение с существующей строкой другой таблицы: задача ссылается на тег, расход — на категорию, книга — на жанр. Такая связь не позволяет сохранить ссылку на отсутствующий справочник.

NOT NULL запрещает отсутствие обязательного значения, UNIQUE запрещает дубликаты, CHECK проверяет простое предметное правило. Ограничения работают независимо от интерфейса, поэтому защищают данные и от ошибочного запроса. Валидация формы остаётся нужна для быстрого и понятного сообщения пользователю.

Изменения схемы записываются в SQL-файл миграции, а начальные справочные строки — в seed.sql. Это позволяет воспроизвести базу в новом проекте: сначала применить миграцию, затем seed. На базовом маршруте курса SQL выполняется через SQL Editor; Supabase CLI можно подключить дополнительно.

### Публичный клиент и реальная защита

Frontend использует URL проекта и publishable key. Этот ключ попадает в JavaScript и виден пользователю; он идентифицирует приложение, но не даёт административных полномочий. Реальный доступ определяют права таблицы и политики RLS. Secret key и legacy service_role обходят RLS, поэтому их нельзя помещать во frontend, VITE-переменные, репозиторий или скриншоты.

Для обращения к таблице должны одновременно разрешить операцию обычные права PostgreSQL и политика RLS. В этой работе роли anon и authenticated получают только SELECT справочника. У личной таблицы RLS включён, но прав и политик пока нет. Поэтому браузер не может читать или изменять личные строки даже при известном адресе проекта.

## Практическая часть

### Шаг 1 Подготовьте проект и аккаунт Supabase

Сохраните завершённую ЛР 4 в Git. Из frontend выполните npm ci и npm run build. Проверьте загрузку справочника из JSON, ошибку, пустой ответ и повтор без потери черновика. Зафиксируйте рабочее состояние отдельным коммитом до изменения источника данных.

Создайте учебный проект Supabase. Выберите понятное имя и надёжный пароль базы, сохраните пароль в менеджере паролей. Регион выбирайте ближе к пользователям. Не публикуйте пароль, access token аккаунта или секретные ключи. Для этой лабораторной достаточно Dashboard, SQL Editor и подключения через библиотеку supabase-js.

В корне репозитория создайте каталоги supabase/migrations и файл supabase/seed.sql. Имя первой миграции включает время и назначение, например supabase/migrations/202609210001_initial_schema.sql. Используйте своё фактическое время; после сдачи не переименовывайте применённую миграцию.

### Шаг 2 Спроектируйте таблицы своей темы

Запишите 2–3 таблицы, их ключи и связь. Основная таблица содержит поля формы ЛР 3, технические id, owner_id и created_at. Справочник содержит стабильный id и отображаемое name. Идентификаторы справочника сохраняйте такими же, как в JSON ЛР 4, чтобы существующие демонстрационные записи и форма продолжили работать.

1. Task Tracker: tasks → tags.
2. Mini Issue Tracker: issues → issue_types; comments появятся в ЛР 7.
3. Expense Tracker: transactions → categories.
4. Фильмы и сериалы: media_entries → genres.
5. Игры: games → platforms; жанр можно оставить ограниченной строкой.

6. Планировщик мероприятий: events → event_categories; registrations — третья таблица.
7. Habit Tracker: habits → habit_categories; checkins — третья таблица.
8. Учебный менеджер: assignments → courses.
9. Библиотека: books → genres.
10. Personal CRM: contacts → companies; interactions появятся в ЛР 7.

Для типов issue достаточно двух строк; для остальных справочников подготовьте минимум пять. Не создавайте отдельную таблицу для каждого текстового поля. Например, автор книги и место события остаются текстом. Справочник выбирают там, где набор значений повторяется и должен управляться отдельно.

Сделайте короткую схему в README или docs/progress.md: название таблицы, назначение, ключ и связь. Отметьте, какие данные публичны, а какие принадлежат пользователю. Для Task Tracker публичны tags, личными будут tasks.

**Шаг 3 Создайте миграцию на примере Task Tracker**

В файл первой миграции внесите SQL для своей темы. Следующий пример создаёт справочник тегов и личные задачи. Названия полей согласованы с предыдущими лабораторными; tag во frontend соответствует tag_id в БД и будет преобразован при переносе задач в ЛР 6.

```
create table public.tags (
  id text primary key,
  name text not null unique,
  constraint tags_id_not_blank
    check (id = btrim(id) and id <> ''),
  constraint tags_name_not_blank
    check (name = btrim(name) and name <> '')
);

create table public.tasks (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references auth.users(id) on delete cascade,
  tag_id text not null references public.tags(id)
    on update cascade on delete restrict,
  title text not null,
  description text not null default '',
  status text not null,
  due_date date not null,
  priority text not null,
  created_at timestamptz not null default now(),
  constraint tasks_title_length
    check (char_length(btrim(title)) between 3 and 100),
  constraint tasks_status_allowed
    check (status in ('todo', 'in_progress', 'done')),
  constraint tasks_priority_allowed
    check (priority in ('low', 'medium', 'high'))
);
```

Адаптируйте названия и CHECK к своей теме: положительная сумма, допустимая оценка, конец события после начала. Не добавляйте тестового owner_id: он должен ссылаться на реального пользователя auth.users, а аккаунты появятся в следующей работе. FK уже существует, хотя таблица основной сущности пока пуста.

Даты храните типом date, когда время суток не нужно, и timestamptz для момента события. Денежные суммы храните numeric, а не float. Оценку, которая может отсутствовать, разрешите как NULL и проверяйте только непустое значение, например rating is null or rating between 1 and 10.

**Шаг 4 Настройте RLS и минимальные права**

Продолжите тот же файл миграции. Сначала включите RLS на обеих таблицах и уберите возможные права клиентских ролей. Затем верните только чтение публичного справочника и создайте политику SELECT. У личной таблицы в ЛР 5 нет клиентских прав и политик.

```
alter table public.tags enable row level security;
alter table public.tasks enable row level security;

revoke all on table public.tags from anon, authenticated;
revoke all on table public.tasks from anon, authenticated;

grant select on table public.tags to anon, authenticated;

create policy "tags are publicly readable"
on public.tags
for select
to anon, authenticated
using (true);
```

Замените tags и tasks именами своих таблиц. Политика using (true) подходит только справочнику, который действительно разрешено видеть всем. Не создавайте policy for all и не выдавайте insert, update или delete справочника. Не создавайте политику tasks using (true): это открыло бы личные данные.

RLS и GRANT решают разные задачи. Политика не возвращает право, отозванное REVOKE, а выданное право не обходит включённый RLS. Храните настройку таблицы, прав и политики рядом в одной миграции, чтобы при повторном создании не забыть часть защиты.

### Шаг 5 Подготовьте воспроизводимые справочные данные

Перенесите значения из JSON ЛР 4 в supabase/seed.sql. В seed находятся только INSERT или UPSERT справочных данных; таблицы, ограничения и политики остаются в миграции. Для Task Tracker используйте те же id:

```
insert into public.tags (id, name)
values
  ('study', 'Учёба'),
  ('work', 'Работа'),
  ('home', 'Дом'),
  ('health', 'Здоровье'),
  ('personal', 'Личное')
on conflict (id) do update
set name = excluded.name;
```

UPSERT делает повторное применение seed предсказуемым и сохраняет стабильные id. Не помещайте в seed реальные персональные данные, email, пароли или строки, принадлежащие пользователям. Демонстрационные задачи ЛР 2 остаются в frontend/src/data и не загружаются в Supabase.

В SQL Editor выполните полный файл миграции один раз, затем seed.sql. Не копируйте случайные отдельные команды без сохранения в репозитории. Если допустили ошибку, исправьте SQL-файл и учебный проект согласованно; запишите в progress.md фактический порядок применения.

**Шаг 6 Проверьте схему и ограничения в SQL Editor**

Откройте Table Editor и убедитесь, что справочник заполнен, а основная таблица пуста. Проверьте типы столбцов, PK и FK. Затем выполните безопасные проверки в SQL Editor и удалите временные строки, если они создавались. Ошибка ограничения считается ожидаемым результатом проверки.

```
select id, name from public.tags order by name;

-- Должна завершиться ошибкой внешнего ключа:
insert into public.tasks (
  owner_id, tag_id, title, status, due_date, priority
) values (
  gen_random_uuid(), 'missing', 'Проверка связи',
  'todo', current_date, 'medium'
);
```

В примере ошибка может возникнуть уже на owner_id, потому что случайного пользователя нет. Это тоже подтверждает FK, но проверку tag_id удобнее выполнить в ЛР 6 с реальным тестовым аккаунтом. Сейчас отдельно проверьте ограничения справочника: пустой id, пустое name и повтор id должны отклоняться.

Не отключайте ограничения ради успешной вставки. Не создавайте фиктивную строку в auth.users вручную. После проверок таблица основной сущности должна остаться пустой, а справочник — содержать только значения seed.

### Шаг 7 Установите клиент и настройте окружение

Из каталога frontend установите официальный клиент. Команда изменит package.json и lock-файл:

```
npm install @supabase/supabase-js
```

В Dashboard откройте Connect или Settings → API Keys. Скопируйте URL проекта и publishable key. Создайте frontend/.env.local со своими значениями и frontend/.env.example с пустыми образцами:

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_your_key
```

В .env.example оставьте понятные заглушки, а не действующие значения. Проверьте .gitignore: .env.local и другие локальные env-файлы не попадают в Git, но .env.example разрешён. Переменные VITE видны в собранном frontend; сюда допустим только publishable key. Secret key, service_role и пароль базы запрещены.

После создания или изменения env-файла перезапустите Vite. Значения подставляются при запуске и сборке. Если переменная отсутствует, приложение должно сообщить об ошибке настройки, а не обращаться по адресу undefined.

### Шаг 8 Создайте единый Supabase-клиент

Создайте frontend/src/lib/supabase.ts. Один экземпляр клиента будет использоваться сервисами frontend. Проверка конфигурации помогает сразу увидеть неверно заполненный env-файл.

```
import { createClient } from '@supabase/supabase-js'

const url = import.meta.env.VITE_SUPABASE_URL
const publishableKey = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY

if (!url || !publishableKey) {
  throw new Error('Не настроены переменные Supabase в .env.local')
}

export const supabase = createClient(url, publishableKey)
```

Не создавайте новый клиент внутри компонента или при каждом запросе. Не выводите ключи в console.log. Наличие publishable key в браузере ожидаемо, но оно безопасно только вместе с правильно настроенными правами и RLS.

**Шаг 9 Замените JSON-сервис запросом к Supabase**

В services/tags.ts сохраните тип Tag и функцию parseTags из ЛР 4. Удалите tagsUrl и прямой fetch к JSON. Импортируйте общий клиент и замените только getTags. Благодаря прежнему контракту useTags и TaskForm менять почти не придётся.

```
import { supabase } from '../lib/supabase'

export async function getTags(signal: AbortSignal): Promise<Tag[]> {
  const { data, error } = await supabase
    .from('tags')
    .select('id, name')
    .order('name')
    .abortSignal(signal)

  if (signal.aborted) {
    throw new DOMException('Запрос отменён', 'AbortError')
  }
  if (error) {
    throw new Error(`Не удалось загрузить теги: ${error.message}`)
  }
  return parseTags(data)
}
```

Замените имя таблицы для своей темы. Явно выбирайте только нужные столбцы. Supabase возвращает пару data/error, поэтому проверьте error до передачи результата. parseTags остаётся полезной проверкой внешних данных, даже если TypeScript знает ожидаемую форму запроса.

useTags из ЛР 4 уже передаёт AbortSignal, различает loading, error и success и умеет повторять запрос. Сохраните его очистку и черновик формы. Добавьте доступную после успеха кнопку «Обновить справочник», которая вызывает retry и не отправляет форму:

```
<button type="button" onClick={retry}
  disabled={load.status === 'loading'}>
  Обновить справочник
</button>
```

Файл public/demo/tags.json больше не является источником приложения. Удалите его после успешного переключения или явно переместите в документационные примеры; не оставляйте скрытый fallback. Ошибка Supabase должна быть видна, иначе невозможно проверить настройку доступа.

**Шаг 10 Проверьте чтение и запрет записи из браузера**

Запустите frontend и откройте форму. В Network найдите запрос к домену проекта Supabase и убедитесь, что ответ содержит справочник. Выберите значение, создайте локальную запись и откройте редактирование. id из БД должен сохраняться в поле tag, а пользователю показывается name.

Временно измените имя таблицы в сервисе и убедитесь, что форма показывает ошибку и повторную попытку. Верните правильное имя. Затем очистите строки справочника через SQL, получите отдельное пустое состояние, снова примените seed.sql и нажмите «Обновить справочник». Перезагрузка приложения для этого не нужна.

Из SQL Editor измените только отображаемое имя одной строки, например:

```
update public.tags set name = 'Учебные задачи' where id = 'study';
```

Нажмите «Обновить справочник» и увидьте новое name при прежнем id. Черновик формы должен сохраниться. После проверки верните согласованное название в seed.sql либо оставьте новое и обновите seed, чтобы удалённая база и репозиторий описывали один результат.

Проверьте запрет клиентской записи. Временно выполните из кода frontend запрос insert в справочник с publishable key. Ожидается ошибка доступа, а строка не появляется. Удалите проверочный код до коммита. Также попробуйте select основной таблицы: браузер не должен получить личные строки. Не добавляйте разрешающие политики ради прохождения этой проверки.

**Шаг 11 Проверьте проект по отдельным слоям**

1. Схема: две или три таблицы созданы из сохранённого SQL; PK, FK, NOT NULL и CHECK соответствуют теме.
2. Данные: seed повторно заполняет тот же справочник без новых id.
3. Доступ: anon и authenticated читают справочник, но не изменяют его; личная таблица закрыта.
4. Конфигурация: без .env.local видна понятная ошибка настройки; после возврата файла приложение запускается.
5. Интерфейс: loading, error, empty и success различимы; retry и обновление не стирают черновик.
6. Непрерывность: создание, редактирование, отмена, удаление и фильтр ЛР 3 работают; основные записи пока локальны.
7. Источник: Network показывает Supabase, а public/demo JSON больше не подменяет результат.
8. Сборка: npm run build проходит с проверкой TypeScript.

Если форма возвращает пустой массив без ошибки, проверьте наличие строк, права SELECT и политику справочника. Если видите 42501 или permission denied, проверьте GRANT до изменения политики. Если чтение работает, но клиентская запись тоже проходит, немедленно проверьте REVOKE и отсутствие лишней политики INSERT или FOR ALL.

**Шаг 12 Сохраните воспроизводимый результат**

Обновите README: укажите Supabase как источник справочника, перечислите имена переменных без значений и сохраните пояснение о временных основных записях. В docs/progress.md добавьте ЛР 5, порядок применения миграции и seed, сценарии чтения, ошибки, пустого результата и запрета записи.

Из frontend выполните npm run build и npm run preview. В preview проверьте форму и запрос к Supabase. В корне убедитесь, что Git видит миграцию, seed, клиент, сервис, package.json, lock-файл и .env.example, но не видит .env.local.

```
git status
git add .
git diff --cached --stat
git commit -m "Lab 5: add Supabase schema and reference data"
git push
```

В чистой копии выполните npm ci и npm run build из frontend. Для локального запуска создайте .env.local по образцу и подставьте настройки учебного проекта. Преподавателю передайте репозиторий и коммит ЛР 5; пароль базы и секретные ключи не передавайте.

## Частые проблемы

Форма пишет, что переменные не настроены. Проверьте точные имена с префиксом VITE_, расположение файла внутри frontend и перезапуск dev-сервера. Не добавляйте fallback с ключом в исходный код.

Запрос возвращает permission denied. Проверьте, что RLS включён, роли получили grant select и существует select policy для справочника. GRANT и политика нужны одновременно. Не выдавайте лишние права для устранения одной ошибки.

Справочник читается, но порядок меняется. Добавьте order по отображаемому имени. Порядок строк таблицы без ORDER BY не гарантируется и не должен зависеть от последовательности seed.

Форма не выбирает значение демонстрационной записи. Сравните id в seed, JSON ЛР 4 и поле tag исходных данных. В option value хранится id, а name используется только для показа. Не исправляйте проблему хранением name вместо связи.

Изменение в Dashboard исчезает после воспроизведения базы. Источником схемы служит миграция, а справочных строк — seed.sql. Любое постоянное изменение нужно записать в соответствующий файл, затем согласованно применить.

В репозиторий попал .env.local. Удалите файл из индекса Git, оставив его на компьютере, исправьте .gitignore и проверьте историю до публикации. Если туда попал secret key или service_role, отзовите ключ в Dashboard; простого удаления файла недостаточно.

## Использование ИИ при выполнении работы

Можно попросить ИИ объяснить конкретное ограничение SQL, ошибку Supabase или различие между GRANT и политикой. Передавайте только структуру таблицы, сообщение об ошибке и обезличенный код. Не передавайте пароль базы, secret key, service_role, токены пользователя или содержимое .env.local. Самостоятельно проверьте SQL и доступ из браузера.

## Результаты работы и сдача

В репозитории находятся миграция схемы и доступа, seed.sql, Supabase-клиент, обновлённый сервис, .env.example и инструкции. В удалённой базе есть связанная схема и публичный справочник. Форма читает его из Supabase, обновляет по кнопке и сохраняет черновик при ошибке. Клиент не может изменять справочник или читать личную таблицу.

На демонстрации примените или покажите SQL, сравните seed со строками БД, откройте форму и запрос в Network. Измените name через SQL и обновите справочник. Покажите пустое состояние и восстановление seed, затем ожидаемую ошибку клиентской записи. Объясните, почему publishable key допустим в браузере, а secret key — нет.

## Критерии оценки

Максимум — 10 баллов. Работа не засчитывается как завершённая, если форма продолжает читать JSON ЛР 4, схема не воспроизводится по SQL или клиенту разрешена анонимная запись.

Функционал — 5 баллов: связанная схема из 2–3 таблиц с ограничениями (1); миграция и воспроизводимый seed (1); корректные RLS, права и политика чтения справочника (1); Supabase-клиент и запрос через сервис (1); справочник используется формой создания и редактирования с обновлением (1).

Ошибки и безопасность — 2 балла: loading, error и empty сохранены, ответ проверяется, повтор не стирает черновик (1); личная таблица закрыта, клиентская запись справочника запрещена, секреты отсутствуют во frontend и Git (1).

Непрерывность проекта — 1 балл: локальный CRUD, маршруты, отмена и фильтр предыдущих работ работают; временный характер основных записей объяснён.

Git и воспроизводимость — 2 балла: миграция, seed, зависимости и коммит доступны преподавателю (1); README, progress.md и .env.example актуальны, чистая копия собирается и запускается после локальной настройки (1).

## Вопросы для защиты

1. Чем primary key отличается от foreign key?
2. Зачем проверять поля и в форме, и ограничениями БД?
3. Почему id справочника должен оставаться стабильным при изменении name?
4. Чем GRANT отличается от политики RLS?
5. Почему личная таблица закрыта до ЛР 6?
6. Почему publishable key можно использовать в браузере, а secret key нельзя?
7. Зачем хранить миграцию и seed, если таблицы уже созданы в Dashboard?
8. Как вы доказали, что форма читает Supabase и не использует старый JSON?

## Материалы для самостоятельной работы

Supabase — React и создание клиента: https://supabase.com/docs/guides/getting-started/tutorials/with-react

Supabase — API keys: https://supabase.com/docs/guides/getting-started/api-keys

Supabase — Row Level Security, права и политики: https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase — миграции: https://supabase.com/docs/guides/deployment/database-migrations

Supabase — seed-данные: https://supabase.com/docs/guides/local-development/seeding-your-database

Supabase JavaScript — select: https://supabase.com/docs/reference/javascript/select

Supabase JavaScript — отмена запроса: https://supabase.com/docs/reference/javascript/using-modifiers-abortsignal

Документация проверена при подготовке работы. Интерфейс Dashboard может меняться; ориентируйтесь на разделы Connect, API Keys и SQL Editor своего проекта.